# The League — how our bot trains itself

Patient Wolves, Battlecode 2026a, branch `league`

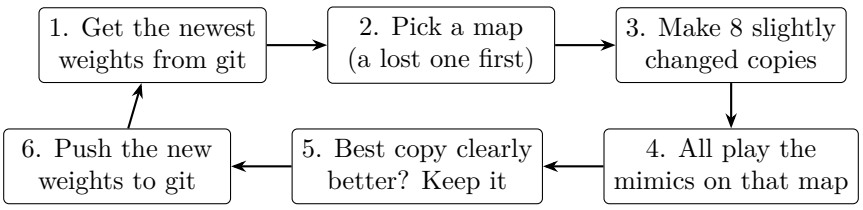
October 2026

## 1 The idea in five sentences

1. We copy the 10 best teams on the ladder into practice bots, called **mimics**.
2. Our bot plays thousands of games against these mimics on every map.
3. After each batch of games we try small changes to our bot's numbers, and keep the changes that win more.
4. When we lose on the real ladder, the bot trains on that map first, and a better version can be uploaded.
5. Anything the training cannot fix is written in a report for a human to look at.

Every teammate's PC runs the same program (`harness/league.py`). They share the results through git.

## 2 One training round




A round takes about 4 minutes. Every third round, we train one of the *mimics* instead of our bot (see §4).

## 3 What the bot learns

Most of the bot is hand-written Rust code: walking, fighting moves, shopping and so on. Training does **not** change that code. It only changes a list of **102 numbers** that the code reads when it makes decisions. Each map has its own copy of the 102 numbers, because a good choice on one map can be bad on another.

| Group | How many | What the numbers control (in plain words) |
|---|---|---|
| **W** – multipliers | 18 | Turn hand-picked settings up or down (1.0 = as written by us): how much we value gems vs gold in a camp; how much walking distance and clearing time count; how strong a camp may be before we avoid it; whether to clear camps on the way somewhere; how big an advantage we want before fighting; at what HP we retreat; how aggressive we are; how much gold before going home to shop; how many gems we carry before delivering them (tribute min/max); at what HP we go help a teammate. |
| **TARGET** – camp choice | 10 | Which camp to go for next. Looks at: gems and gold in the camp, distance, time to clear it, time until it respawns, how many others want it, damage we would take, how full our bag is, how many emblems we have, and the game round. |
| **FIGHT** – fight or not | 9 | Whether a fight looks good. Looks at: our HP, enemy HP, allies near, enemies near, whether we or they are ranged, how full our bag is, distance to our spawn. |
| **U** – what to do now | 60 | A small scorer that chooses between 6 actions each turn: keep going, step back, support an ally, fight, walk home, recall. It looks at 10 facts: our HP, damage from camps, damage from enemies, enemies near, allies near, an ally in trouble, bag size, distance to spawn, ranged enemy, plus a constant. $6 \times 10 = 60$ numbers. |
| **X** – extras | 4 | Whether Tavi re-buys stats, and how much fuel to keep spare before using the scorer above (two slots are unused). |
| **HOT** | 1 | Avoid places where we died before. Switched off (it made us worse). |

**Not learned (hand-written or fixed per map):** pathfinding, the draft (which champions we pick), emblem build order, the shopping list, and spell use. Changing these means a human edits the code in `crates/`. The list of 102 is in `harness/params.py`; their defaults are in `bots/main/src/learned.rs`.

**How a change is judged.** A bot's score on a map is

$$\text{score} = \text{win rate} + 0.25 \times \text{gem lead} - 0.01 \times \text{deaths}.$$

A changed copy is kept only if (a) its score is at least 0.04 higher than the current bot on the same games, (b) it beats the current bot head-to-head, (c) it also wins against 3 other opponents it wasn't trained on, and (d) it doesn't run out of fuel more often. Games are deterministic, so both bots are compared on exactly the same games.

## 4 Who we train against

| | |
|---|---|
| **Faithful mimic** (`mimic-nyx`) | A copy of a real team. From their recent ladder games we copy their champion picks, emblem builds, first camps on each map, and how often they deliver gems and shop. Their fighting can't be copied (the game data doesn't show it), so a mimic fights like our own bot. Never changed by training, so "our win rate vs Nyx" stays an honest number. |
| **Evolved mimic** (`mimx-nyx`) | The same copy, but its numbers are trained to *beat us*. Its picks, builds, first camps and delivery habits stay locked to the real team. If the real teams stop updating their bots, these keep getting harder, so our training doesn't stall. If the real team changes its bot, its evolved mimic starts over. |
| **Anchor bots** (`claude-*`, `exploit-*`, `baseline`) | Hand-written bots that attack known weak spots (for example killing our carrier near the altar). They stop old bugs coming back. |

We pick opponents that beat us more often than ones we already beat.

## 5 When a new version goes on the ladder

Only one PC, the **lead**, talks to the platform. If it runs with `--auto-submit`, it uploads the newest trained version when **all** of these are true:

- the trained version is newer than the one on the ladder;
- at least 6 hours have passed since the last upload;
- we lost a ladder game since then, **or** the new version wins at least 5% more against the mimics.

If the new version then does more than 10% worse on the ladder (after 10 ranked games each), the previous version is put back.

## 6 What humans do

Every few hours the lead writes `league/review/<date>.md`. Read it, and write your thoughts in its *Reviewer notes* section. The program keeps that section.

**The "gap" column** compares our win rate against a team's mimic with our real win rate against that team. A big gap means the mimic is too easy, because the real team does something we didn't copy. Watch their replays.

**Losses training couldn't fix** (after 6 tries on that map) are listed with a hint:

- *"mimic can't express it"*: we beat their mimic but lost to the real team. Fix the mimic.
- *"model can't express it"*: we keep losing even to the mimic. None of the 102 numbers can fix it, so the bot needs a new feature in the code. Do it on a `feat/` branch, test it, and merge it into `league`. Every PC picks it up on its next round.

## 7 Maps

The lead follows the platform's current map pool (21 maps in October 2026). New maps are downloaded into `maps/`, and one test game per map records its ID and layout. Maps that leave the pool move to `maps/retired/` and are no longer trained on. Losses on a map we didn't have wait until it arrives.

## 8 The files that matter

| | |
|---|---|
| `league/champion.json` | Our best numbers for each map, and the history of improvements. Shared by all PCs. |
| `bots/main/src/pack/pack.bin` | The same numbers in the format the bot reads. Rebuilt automatically. |
| `league/mimics/<team>.json` | The evolved mimics' numbers. |
| `league/ledger.json` | Ladder data: top 10, our games, lost games, uploads. Written by the lead. |
| `data/mimic/` | The copied teams. |
| `league/review/` | Reports for humans. |
| `runs/league/` | This PC's logs and scratch files. Safe to delete. |

Never edit the `league/*.json` files or `pack.bin` by hand. If two PCs improve the bot at the same time, the program merges their work. On the same map, a head-to-head game decides which change wins.

## 9 Commands

**Set up a new PC** (needs cambc 3.0.1 or newer, the same engine as the ladder):

```
git clone -b league https://github.com/Ahmed-Abdul-Rahim/battlecode-2026a.git
cd battlecode-2026a
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1     (Linux/macOS: bash scripts/
    setup.sh)
.\.venv\Scripts\Activate.ps1
cambc login                       (lead PC only)
```

**Run it** (Linux/macOS: `scripts/train.sh` with the same words):

```
scripts\train.ps1 start                          normal PC: train and share
scripts\train.ps1 start --lead                   the one lead PC: also reads the ladder
scripts\train.ps1 start --lead --auto-submit     lead that may also upload (only one PC!)
scripts\train.ps1 status                         what is happening
scripts\train.ps1 logs                           live log
scripts\train.ps1 stop
scripts\train.ps1 once --no-push                 one round, no git (for testing)
scripts\train.ps1 sync | maps | bench | review   run one lead step now
scripts\train.ps1 submit                         upload by hand (agree with the team first)
```

**Look at games:**

```
bctools run main mimic-nyx --maps 00lanes       play our bot vs a mimic
bctools gate main                               quick check vs the baseline bot
cambc view <replay>.repl26a                     watch a replay
cambc matches download <matchId> -o x.repl26a   get a ladder game from the report
```

**Rebuild this PDF:** `cd docs; latexmk -pdf league.tex; latexmk -c`

## 10 When something goes wrong

| | |
|---|---|
| “already running” | Run `scripts\train.ps1 stop`, then start again. |
| “maps pull failed” | cambc is too old. Run `pip install -U "cambc>=3.0.1"`, then `cambc init --yes .` |
| “learned.rs and params.py disagree” | Someone changed the list of numbers. Update `harness/params.py` to match. |
| “build failed” | A broken code change was pulled. Fix it on a branch. |
| “merge needs a human” | Two people changed the same code. Fix the git conflict by hand. |
| “no mimics yet” | The lead hasn’t synced yet. Training uses the anchor bots meanwhile. |
| Results differ from the ladder | Check that your cambc is 3.0.1 or newer. |
| An evolved mimic looks broken | Delete `league/mimics/<team>.json` and push. It starts over. |